MANUAL PENGGUNA DAN PENERAPAN

PASUPATASTRA

Unified Command Center For Multi-drone Operations

Ground Control Station multi-drone: konsol operator, server, bridge DJI dan Autel, video, dan analitik AI

Versi perangkat lunak 0.3.0 · DarmawanRobotics / nextgen-gcs

Dokumen internal. Isi dapat berubah mengikuti pengembangan.

Daftar Isi

1 Pendahuluan **4**

1.1 Komponen . . . 4

1.2 Istilah . . . 4

1.3 Status dokumen . . . 5

2 Arsitektur **6**

2.1 Alur data . . . 6

2.2 Port jaringan . . . 7

3 Penerapan server **8**

3.1 Kebutuhan . . . 8

3.2 Instalasi satu kali . . . 8

3.3 Build konsol . . . 8

3.4 Menjalankan . . . 8

3.5 Akun operator . . . 9

3.6 Server Linux dengan GPU NVIDIA . . . 9

3.7 Akses jarak jauh . . . 9

3.8 Pemeliharaan . . . 9

4 Bridge drone **10**

4.1 Protokol singkat . . . 10

4.2 Batas keselamatan . . . 10

4.3 Bridge DJI Matrice 4T . . . 10

4.3.1 Prasyarat . . . 10

4.3.2 Build . . . 10

4.3.3 Konfigurasi di aplikasi . . . 11

4.3.4 Perilaku perintah (MSDK v5) . . . 11

4.4 Bridge Autel EVO II V3 . . . 11

4.4.1 Prasyarat . . . 12

4.4.2 Build . . . 12

4.4.3 Konfigurasi di aplikasi (menu Pengaturan) . . . 12

4.4.4 Perilaku perintah (Autel SDK v1.5) . . . 12

4.5 Mendaftarkan drone di konsol . . . 12

5 Video dan kamera **13**
5.1 Mengirim video . . . 13
5.2 Mendaftarkan sumber . . . 13
5.3 Video beranotasi . . . 13

6 Panduan konsol **14**
6.1 Masuk . . . 14
6.2 Terbang . . . 14
6.3 Misi . . . 14
6.4 Titik drop . . . 14
6.5 Peta armada . . . 14
6.6 Video . . . 15
6.7 Drone dan Kamera . . . 15
6.8 Model AI . . . 15
6.9 Log dan Pengaturan . . . 15

7 Analitik AI **16**
7.1 Deteksi multi-model . . . 16
7.2 Hitungan grup dan prompt . . . 16
7.3 Caption dan tanya-jawab visual . . . 16
7.4 Speech-to-text . . . 17

8 Keselamatan dan prosedur uji terbang **18**
8.1 Lapisan keselamatan . . . 18
8.2 Daftar periksa sebelum terbang . . . 18
8.3 Urutan uji lapangan pertama . . . 18
8.4 Jika terjadi masalah . . . 19

9 Performa **20**
9.1 Ringkasan . . . 20
9.2 Latensi REST . . . 20
9.3 Telemetri armada . . . 20
9.4 Batasan pengukuran . . . 21
9.5 Mengukur ulang di server produksi . . . 21

10 Pemecahan masalah **22**

11 Fitur saat ini dan rencana **23**
11.1 Tersedia di versi 0.3.0 . . . 23
11.2 Rencana . . . 23

A Protokol bridge **24**

A.1 Endpoint 24
A.2 Pesan 24
A.3 Telemetri 24
A.4 Jenis perintah 24

B Referensi **25**
B.1 Variabel lingkungan API 25
B.2 Variabel lingkungan worker 26
B.3 Endpoint API 26
B.4 CLI administrasi 27

BAB 1

Pendahuluan

PASUPATASTRA adalah *ground control station* (GCS) untuk operasi banyak drone dari vendor berbeda dalam satu konsol. Operator melihat posisi dan status seluruh armada di satu peta, menonton video dari drone, ponsel, dan CCTV, menjalankan deteksi objek dan analisis adegan berbasis AI, serta mengirim perintah terbang dan misi waypoint ke drone yang mendukungnya.

Seluruh antarmuka operator berbahasa Indonesia. Seluruh sistem berjalan di server lokal (misalnya Mac Mini M4 di pos komando) tanpa layanan cloud wajib.

1.1 Komponen

Komponen	Fungsi
Konsol (`app/`)	Aplikasi operator Next.js, berjalan di browser dan sebagai aplikasi desktop Tauri.
`gcs-api`	Server FastAPI: autentikasi, registri drone dan kamera, endpoint bridge, perintah dan misi, simulator drone demo, perekaman penerbangan, tanya-jawab visual (VLM), speech-to-text.
`gcs-worker`	Proses inferensi: beberapa model YOLO per stream, hitungan grup, caption Moondream, video beranotasi.
MediaMTX	Server media: menerima RTMP/RTSP/SRT dan menyajikan WebRTC/HLS.
NATS	Bus pesan antara API dan worker.
PostgreSQL + TimescaleDB	Penyimpanan data, kejadian, audit, dan jejak terbang.
Ollama	Model bahasa-visual lokal (Moondream, penerjemah).
Bridge DJI	Aplikasi Android di remote DJI untuk Matrice 4T (MSDK v5.17).
Bridge Autel	Aplikasi Android di Smart Controller V3 untuk EVO II V3 (Autel SDK v1.5).

1.2 Istilah

Bridge Aplikasi di remote controller yang meneruskan telemetri ke server dan menjalankan perintah dari server.

Interlock Saklar **Izinkan kendali dari GCS** di aplikasi bridge. Selama mati, drone hanya mengirim telemetri dan semua perintah ditolak.

Drone demo Wahana simulasi di server, berwarna ungu dengan label DEMO, untuk mendemonstrasikan fitur (termasuk drop payload) tanpa drone fisik.

Stream Satu sumber video: kamera drone atau kamera tetap (CCTV/ponsel).

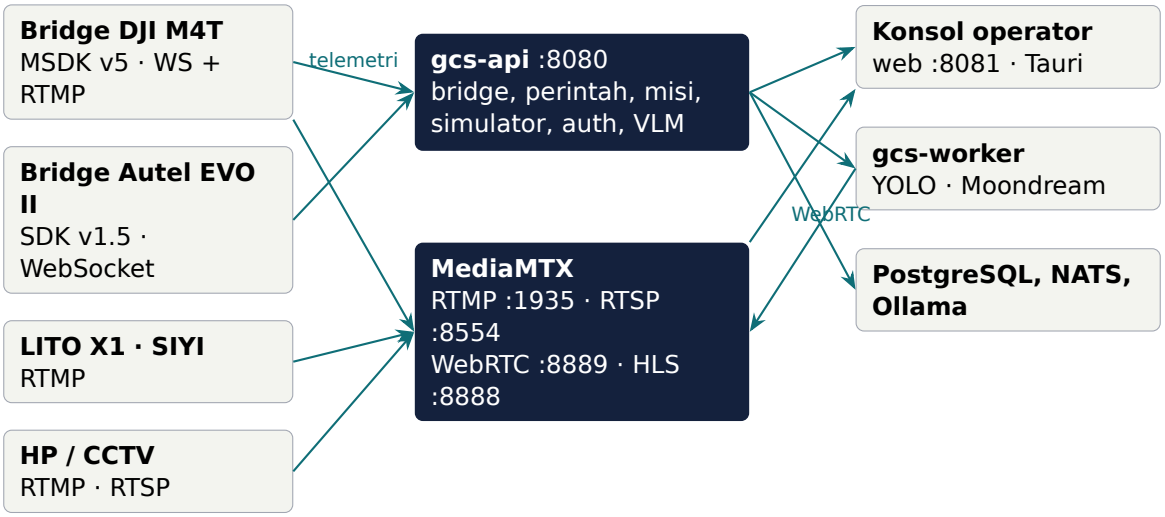
Kunci model Nama pendek unik model deteksi, dipakai pada hitungan `model:kelas`.

1.3 Status dokumen

Manual ini menggambarkan versi 0.3.0. Bagian yang belum diuji di perangkat keras nyata ditandai jelas. Hasil pengujian otomatis dan pengukuran performa ada di Bab 9.

BAB 2

Arsitektur



2.1 Alur data

1. **Telemetri.** Bridge membuka WebSocket ke `gcs-api`. API menormalkan format vendor ke satu model telemetri, mendeteksi lepas landas dan mendarat, mencatat jejak terbang, lalu menyiarkan ke konsol melalui `/ws/realtime` dengan laju 5 Hz per drone.
2. **Perintah.** Konsol memanggil `POST /drones/{id}/commands`. API memeriksa izin, kapabilitas bridge, dan batas keselamatan, lalu mengirim permintaan ke bridge dan menunggu *ack*. Semua perintah dicatat di log audit.
3. **Video.** Drone dan ponsel mengirim RTMP ke MediaMTX. Kamera IP (RTSP/HLS) ditarik oleh MediaMTX ke jalur `src/<id>`. Konsol menonton lewat WebRTC (WHEP).
4. **AI.** API mengirim *desired state* ke worker lewat NATS. Worker membaca RTSP, menjalankan model, menerbitkan deteksi dan caption, dan mempublikasikan video beranotasi ke jalur `ai/<path>`.

2.2 Port jaringan

Port	Protokol	Kegunaan
8080	HTTP/WS	API, WebSocket realtime, endpoint bridge
8081	HTTP	Konsol web + proxy API (Caddy)
1935	TCP	RTMP masuk dari drone, ponsel, CCTV
8554	TCP	RTSP (worker membaca dan menerbitkan)
8889	HTTP	WebRTC WHEP untuk konsol
8189	UDP	Media WebRTC
8888	HTTP	HLS
8890	UDP	SRT masuk

BAB 3

Penerapan server

3.1 Kebutuhan

- Mac Mini M4 (disarankan RAM 16 GB atau lebih) dengan macOS terbaru, IP LAN tetap (contoh di manual: `192.168.1.10`).
- Docker Desktop, Homebrew, Node.js 20+, Python 3.12, Ollama, PM2.
- Router Wi-Fi yang menghubungkan server dengan remote controller drone dan laptop operator.

3.2 Instalasi satu kali

```
git clone <repo> pasupatastra && cd pasupatastra
# ffmpeg, venv Python, ollama pull moondream + llama3.2:3b, pm2
./infra/scripts/setup-mac.sh
cd infra && cp .env.example .env
```

Isi `.env` minimal:

Variabel	Isi
`LAN_IP`	IP LAN server, dipakai sebagai kandidat WebRTC.
`PUBLIC_WEBRTC_URL`	`http://<LAN_IP>:8889`
`POSTGRES_PASSWORD`	Kata sandi database, acak.
`GCS_JWT_SECRET`	String acak panjang (minimal 32 karakter).
`GCS_ADMIN_PASSWORD`	Kata sandi awal akun `admin`; ganti setelah login pertama.
`GCS_BRIDGE_TOKEN`	Opsional. Jika diisi, URL bridge wajib memakai `?token=...`.

3.3 Build konsol

```
cd app && npm ci
NEXT_PUBLIC_API_URL=http://192.168.1.10:8081/api npm run build   # hasil di app/out
```

Tanpa `NEXT_PUBLIC_API_URL`, konsol berjalan dalam *mode simulasi* penuh di browser tanpa server; berguna untuk pelatihan dan demo.

Aplikasi desktop: `npm run tauri build` dengan variabel yang sama menghasilkan installer macOS/Windows.

3.4 Menjalankan

```
cd infra
docker compose up -d --build          # MediaMTX, TimescaleDB, NATS, API, web
pm2 start pm2/ecosystem.config.cjs    # worker AI native (CoreML / VideoToolbox)
pm2 save && pm2 startup
```

Periksa kesehatan: `curl http://localhost:8080/health` harus berisi `"ok": true`, dan `"worker"` tidak `null` setelah worker berjalan.

Buka konsol di `http://192.168.1.10:8081`, masuk sebagai `admin`, lalu ganti kata sandi di **Pengaturan → Akun**.

3.5 Akun operator

Tidak ada pendaftaran mandiri. Administrator membuat akun lewat CLI:

```
docker compose exec api gcs-admin create-user \
    --username op1 --display-name "Operator 1" --role operator
docker compose exec api gcs-admin set-password --username op1
```

Peran: `admin` (semua), `operator` (mengubah data dan mengirim perintah), `observer` (hanya melihat).

3.6 Server Linux dengan GPU NVIDIA

`docker compose --profile gpu up -d` menjalankan worker CUDA dan Ollama di container. Model CoreML (`.mlpackage`) hanya berjalan pada worker macOS.

3.7 Akses jarak jauh

Catatan

Cloudflare Tunnel (`infra/cloudflared/config.example.yml`) hanya meneruskan HTTP: konsol, API, dan WebSocket berfungsi, tetapi media WebRTC (UDP) tidak. Untuk video langsung dari luar LAN gunakan VPN (WireGuard atau Tailscale) ke jaringan pos komando.

3.8 Pemeliharaan

- Cadangan database: `docker compose exec postgres pg_dump -U gcs gcs > backup.sql`
- Pembaruan: `git pull`, build ulang konsol, `docker compose up -d --build`, `pm2 restart gcs-worker`. Migrasi database berjalan otomatis saat API mulai.
- Log: `docker compose logs -f api`, `pm2 logs gcs-worker`.

BAB 4

Bridge drone

Bridge adalah aplikasi Flutter + Kotlin di remote controller. Folder `bridges/dji` dan `bridges/autel` berisi `lib/` (UI Flutter) dan `android/` (kode native dan SDK). Gabungkan ke proyek Flutter masing-masing (yang memiliki `pubspec.yaml`) atau buat proyek baru dengan `flutter create` lalu salin kedua folder tersebut.

4.1 Protokol singkat

Bridge membuka WebSocket ke server, mengirim `hello` berisi kapabilitas, lalu telemetri berkala. Server mengirim `command` atau `mission` dengan `id`, bridge menjawab `ack` dengan `id` yang sama. Detail JSON ada di Lampiran A.

Kapabilitas `commands` dan `mission` hanya diumumkan bila interlock menyala. Konsol menonaktifkan tombol yang tidak didukung dan menampilkan alasannya.

4.2 Batas keselamatan

Dicek dua kali, di server dan di bridge (bridge memakai posisi GPS terbaru):

Batas	Nilai
Ketinggian titik misi	5–120 m (dan tidak melebihi *Ketinggian maks.* drone)
Jarak titik dari posisi drone	maks. 1000 m
Kecepatan	maks. 10 m/s
Jumlah titik	maks. 99

4.3 Bridge DJI Matrice 4T

4.3.1 Prasyarat

- Remote controller M4T yang didukung DJI Mobile SDK v5.17 (periksa daftar perangkat yang didukung di dokumentasi MSDK).
- App key dari `developer.dji.com` untuk paket `com.darmawan.dronebridge`.
- Android Studio, Flutter SDK, JDK 17.

4.3.2 Build

```
cd bridges/dji/android
cp local.properties.example local.properties    # isi sdk.dir, flutter.sdk, DJI_API_KEY
cd .. && flutter build apk --release
adb install -r build/app/outputs/flutter-apk/app-release.apk
```

Kunci API

Kunci DJI dibaca dari `local.properties` (tidak di-commit) dan disisipkan ke manifest saat build. Jangan menulis kunci langsung di `AndroidManifest.xml`.

4.3.3 Konfigurasi di aplikasi

Isian	Nilai
WebSocket URL	`ws://192.168.1.10:8080/telemetry/m4t-001` (akhir URL = ID drone di konsol)
RTMP URL	`rtmp://192.168.1.10:1935/m4t`
Izinkan kendali dari GCS	Mati untuk telemetri saja; nyalakan hanya saat pilot siap mengambil alih. Berlaku saat bridge dimulai.

Tekan **START BRIDGE**. Bridge berjalan sebagai *foreground service* dan tetap hidup saat aplikasi diminimalkan.

4.3.4 Perilaku perintah (MSDK v5)

Perintah	Implementasi dan catatan
Aktifkan/matikan motor	**Tidak didukung terpisah** oleh MSDK v5. Motor menyala saat lepas landas dan mati otomatis setelah mendarat; bridge menjawab dengan penjelasan.
Lepas landas	`KeyStartTakeoff`: naik sekitar 1,2 m lalu hover.
Mendarat	`KeyStartAutoLanding`.
Kembali ke home	`KeyStartGoHome`.
Tahan posisi	Menjeda misi, atau `KeyStopGoHome` / `KeyStopAutoLanding`.
Terbang ke titik	Misi satu titik (KMZ), berakhir hover.
Misi waypoint	KMZ WPML dibuat di aplikasi, diunggah dengan `pushKMZFileToAircraft`, dijalankan dengan `startMission`. Ketinggian relatif titik lepas landas; hilang sinyal RC: kembali.
Lensa	RGB, Termal, atau Split (PIP) tanpa perlu interlock.
Lepas payload	Tidak didukung; butuh integrasi PSDK.

Status verifikasi

Nama dan tanda tangan API DJI dicocokkan dengan sampel resmi `Mobile-SDK-Android-V5`. Kode belum dikompilasi dengan SDK DJI (SDK hanya tersedia dari Maven DJI); build pertama di Android Studio adalah pemeriksaan wajib.

4.4 Bridge Autel EVO II V3

4.4.1 Prasyarat

- Autel Smart Controller V3, EVO II V3.
- App key dari Autel Developer untuk paket `com.darmawanrobotics.autelgcsbridge`.
- SDK `autel-sdk-release.aar` (v1.5) di `android/app/libs/`.

4.4.2 Build

```
cd bridges/autel/android
cp local.properties.example local.properties    # isi sdk.dir, flutter.sdk, AUTEL_APP_KEY
cd .. && flutter build apk --release
adb install -r build/app/outputs/flutter-apk/app-release.apk
```

4.4.3 Konfigurasi di aplikasi (menu Pengaturan)

Isian	Nilai
WebSocket URL	`ws://192.168.1.10:8080/ws/drone`
Drone ID	`evo2-v3-001` (sama dengan ID di konsol)
Telemetry rate	5 Hz
Izinkan kendali dari GCS	Mati bawaan. Menyimpan pengaturan memulai ulang koneksi dan mengumumkan kapabilitas baru.

4.4.4 Perilaku perintah (Autel SDK v1.5)

Perintah	Implementasi dan catatan
Aktifkan/matikan motor	`Evo2FlyController.droneArmed/droneDisarmed`; ditolak saat di udara.
Lepas landas	`takeOff`: naik sekitar 1,2 m lalu hover; ditolak bila GPS belum valid.
Mendarat	`land(false)`.
Kembali ke home	`goHome`.
Tahan posisi	Menjeda misi, atau `cancelReturn` / `cancelLand`.
Terbang ke titik	Misi satu titik `Evo2WaypointMission`, berakhir hover.
Misi waypoint	`MissionManager.prepareMission` lalu `startMission`; hover pada titik dengan waktu tahan; hindar rintangan: hover; hilang sinyal RC: kembali ke home.
Lensa, payload	Tidak didukung untuk EVO II V3.

Status verifikasi

`CommandHandler.kt` dikompilasi terhadap `autel-sdk-release.aar` asli, dan urutan misi mengikuti sampel resmi `AutelSDK/AndroidAdvanceSample`.

4.5 Mendaftarkan drone di konsol

Buka **Drone → Tambah drone**: ID sama dengan bridge, protokol telemetri *WebSocket* (URL boleh kosong karena bridge yang menyambung), sumber video dengan URL RTMP yang sama seperti di bridge. Tombol **Uji tautan** menampilkan apakah bridge sudah terhubung.

BAB 5

Video dan kamera

5.1 Mengirim video

Semua sumber RTMP mengirim ke `rtmp://192.168.1.10:1935/<jalur>`. Jalur pada URL menjadi nama stream di MediaMTX.

Sumber	Pengaturan
DJI M4T	Dari bridge DJI: `rtmp://192.168.1.10:1935/m4t`
LITO X1, SIYI	Isi RTMP di aplikasi vendor: `rtmp://192.168.1.10:1935/live/litox1`
Ponsel sebagai CCTV	Aplikasi siaran RTMP apa pun: `rtmp://192.168.1.10:1935/live/cctv1`
Kamera IP	RTSP/HLS milik kamera; MediaMTX menariknya otomatis.

5.2 Mendaftarkan sumber

Kamera drone didaftarkan sebagai *sumber video* di form drone. Kamera tetap didaftarkan di halaman **Kamera** (nama, protokol, URL, koordinat opsional agar tampil di peta).

Status *Siaran aktif* muncul setelah MediaMTX menerima gambar. Video ditonton lewat WebRTC dengan latensi rendah di LAN.

5.3 Video beranotasi

Bila stream diberi model deteksi, worker menerbitkan video dengan kotak deteksi, hitungan grup, dan caption ke jalur `ai/<jalur>`. Konsol otomatis beralih ke versi beranotasi saat tersedia; jika tidak, kotak digambar di browser.

BAB 6

Panduan konsol

6.1 Masuk

Buka alamat konsol, isi nama pengguna dan kata sandi. Sesi berlaku 12 jam. Mengganti kata sandi mengeluarkan semua sesi lain.

6.2 Terbang

Tampilan instrumen seperti QGroundControl: peta, horizon buatan, data terbang, dan panel aksi di kiri. Pilih drone di bilah atas.

- Aksi (lepas landas, mendarat, RTH, tahan, terbang ke titik) aktif hanya bila link terhubung dan bridge mengizinkan kendali. Arahkan kursor ke tombol abu-abu untuk melihat alasannya.
- Aksi kritis memakai *geser untuk konfirmasi*.
- **Terbang ke titik**: aktifkan mode, klik peta, konfirmasi ketinggian.

6.3 Misi

Klik peta untuk menambah titik; atur ketinggian, kecepatan, waktu tahan, dan aksi (foto, hover, lepas payload untuk drone demo). **Simpan** menyimpan rencana di server. **Unggah ke drone** mengirim ke drone; **Mulai misi** aktif setelah unggahan berhasil. Rencana bisa diekspor/impor sebagai JSON.

6.4 Titik drop

Target drop berupa koordinat nyata yang tersimpan dan tampil di peta. Eksekusi hanya berjalan pada **drone demo** (simulasi): drone menuju titik, stabil, melepas payload, lalu kembali. Drone nyata memerlukan integrasi payload khusus.

6.5 Peta armada

Semua drone (warna menurut status; drone demo ungu), kamera tetap, target drop, jejak terbang saat ini, dan riwayat penerbangan yang bisa diputar.

6.6 Video

- Grid semua stream; label *CCTV* untuk kamera tetap.
- Pemilih model: centang beberapa model sekaligus (maks. 6) untuk satu stream.
- **Mode fokus**: satu stream besar dengan hitungan grup, waktu inferensi per model, pemilih lensa (DJI), umpan caption AI, dan panel tanya-jawab visual (ketik atau gunakan mikrofon).

6.7 Drone dan Kamera

Registri drone (ID, callsign, model, telemetri, sumber video, parameter terbang) dan registri kamera tetap. Saklar **Drone demo (simulasi)** membuat wahana simulasi untuk fitur demo.

6.8 Model AI

Unggah model (`.pt`, `.onnx`, `.engine`, atau `.mlpackage` dalam zip), **Muat** ke worker, lalu **Ubah** untuk mengatur ambang keyakinan, interval frame, filter kelas, dan warna kotak. Perubahan parameter berlaku tanpa memuat ulang model.

6.9 Log dan Pengaturan

Log menampilkan kejadian sistem, kendaraan, AI, dan audit perintah dengan filter. **Pengaturan** berisi akun, satuan, peta, dan preferensi tampilan.

BAB 7

Analitik AI

7.1 Deteksi multi-model

Setiap stream dapat menjalankan beberapa model YOLO bersamaan. Per model:

Parameter	Arti
Kunci	Nama pendek unik; hitungan memakai `kunci:kelas`.
Ambang keyakinan	Deteksi di bawah nilai ini diabaikan.
Tiap N frame	Model dijalankan setiap N frame; di antaranya hasil terakhir dipakai.
Filter kelas	Hanya kelas terpilih yang ditampilkan dan dihitung.
Warna	Warna kotak di video beranotasi.

7.2 Hitungan grup dan prompt

Berkas `infra/worker/prompts.toml` dimuat ulang otomatis saat disimpan.

```
[count_groups]
orang = ["pedestrian", "people", "person"]
kendaraan = ["car", "van", "truck", "bus", "motor"]

[prompts]
crowd = "{orang} people and {kendaraan} vehicles found. What are the people doing?"

[moondream]
active = ["caption", "crowd"]     # bergiliran tiap siklus
interval_sec = 1.0

[translate]
model = "llama3.2:3b"             # caption diterjemahkan ke Bahasa Indonesia
```

Entri `"kelas"` mencocokkan kelas itu di semua model; `"kunci:kelas"` hanya di satu model. Placeholder prompt: nama grup, `{counts}`, `{time}`.

7.3 Caption dan tanya-jawab visual

Caption Moondream hanya berjalan untuk stream yang sedang dibuka di mode fokus (hemat GPU). Tanya-jawab visual memakai `GCS_VLM_MODEL`: `moondream` (cepat, Inggris, diterjemahkan bila `GCS_TRANSLATE_MODEL` diisi) atau VLM multibahasa seperti `qwen2.5vl:3b` yang menjawab langsung dalam Bahasa Indonesia.

7.4 Speech-to-text

Mikrofon di panel tanya-jawab memakai faster-whisper di server (model `small`, CPU). Bila tidak terpasang, konsol memakai pengenal suara browser.

BAB 8

Keselamatan dan prosedur uji terbang

Pilot tetap penanggung jawab

PASUPATASTRA tidak menggantikan pilot. Selama drone dikendalikan dari GCS, pilot memegang remote, melihat drone secara langsung, dan siap mengambil alih kapan saja. Mematikan interlock hanya menolak perintah *baru*; misi yang sedang berjalan dihentikan dari remote (tombol jeda/rem atau RTH sesuai manual vendor) atau tombol **Tahan** di konsol.

8.1 Lapisan keselamatan

1. **Interlock di remote**: kendali dari GCS mati secara bawaan.
2. **Batas misi ganda** di server dan bridge (Bab 4.2).
3. **Konfirmasi geser** untuk aksi kritis di konsol.
4. **Log audit**: setiap perintah beserta pengguna, hasil, dan pesan.
5. **Peran pengguna**: akun pengamat tidak dapat mengirim perintah.
6. **Failsafe vendor**: hilang sinyal RC memicu kembali ke home (diatur di misi).

8.2 Daftar periksa sebelum terbang

- Area terbuka tanpa orang di bawah jalur, izin terbang sesuai ketentuan setempat.
- Pilot memegang remote, satu pengamat visual, komunikasi dengan operator konsol.
- Baterai drone dan remote di atas 60 %, GPS minimal 12 satelit, titik home tercatat.
- Ketinggian RTH di aplikasi vendor di atas rintangan tertinggi; batas ketinggian drone diatur.
- Konsol: drone *terhubung*, posisi, arah, dan baterai sama dengan aplikasi vendor.
- Interlock **mati** sampai pilot menyatakan siap.

8.3 Urutan uji lapangan pertama

Lakukan berurutan; lanjut hanya bila tahap sebelumnya berhasil.

Tahap	Uji	Langkah dan hasil yang diharapkan
0	Telemetri saja	Interlock mati. Gerakkan drone di darat/terbang manual; konsol mengikuti. Tombol aksi abu-abu dengan alasan *Kendali belum diizinkan*.
1	Arm/disarm (Autel)	Interlock menyala, drone di darat. **Aktifkan motor** → baling-baling berputar; dalam 5 detik **Matikan motor**. Pada DJI, perintah ini ditolak dengan penjelasan (perilaku benar).
2	Lepas landas pendek	**Lepas landas** → drone naik sekitar 1,2 m dan hover. Tunggu 10 detik, **Mendarat**. Periksa kejadian di Log.
3	RTH	Pilot menerbangkan manual ke 15–20 m tinggi dan sekitar 20 m jauhnya. **Kembali ke home** → drone naik ke ketinggian RTH, kembali, dan mendarat.
4	Tahan	Saat RTH berjalan, tekan **Tahan** → RTH dibatalkan, drone hover.
5	Waypoint dekat	Rencana 2–3 titik, ketinggian 20–30 m, jarak kurang dari 100 m, kecepatan 3 m/s, akhir: kembali ke home. Lepas landas dulu, lalu **Unggah ke drone** dan **Mulai misi**. Uji **Jeda** dan lanjutkan. Kejadian *Titik n tercapai* muncul di Log.

8.4 Jika terjadi masalah

- Drone bergerak tidak sesuai: pilot segera ambil alih dengan remote (jeda/rem, lalu stik manual).
- Konsol kehilangan link: drone tetap mengikuti perintah terakhir dan failsafe vendor; kendalikan dari remote.
- Catat waktu, lalu ekspor kejadian dari halaman Log untuk evaluasi.

BAB 9

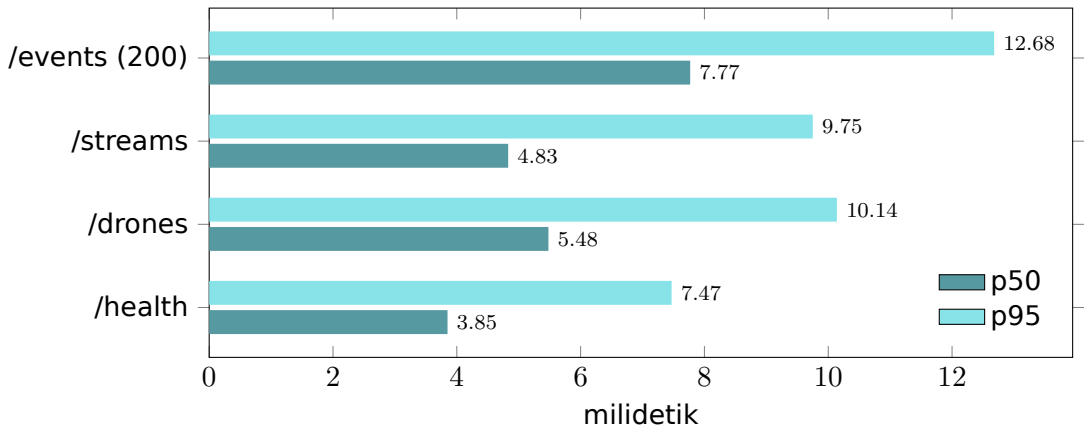
Performa

Angka di bab ini diukur dengan `backend/tests/bench.py` terhadap stack yang benar-benar berjalan (API, worker, NATS, MediaMTX, PostgreSQL) di mesin uji **1 vCPU Xeon 2,1 GHz, RAM 3 GB**. Mac Mini M4 jauh lebih kuat; ulangi pengukuran di sana untuk angka produksi.

9.1 Ringkasan

Pengukuran	Hasil	Keterangan
Login (bcrypt)	615 ms	sengaja lambat, menahan tebakan sandi
Ingest telemetri	196 pesan/s	20 drone × 10 Hz selama 15 s
CPU / RAM proses API	5.3 % / 76 MB	selama beban di atas
Bridge → layar (p50 / p95)	45 / 98 ms	termasuk jeda siaran 5 Hz
Perintah → ack (p50 / p95)	11 / 21 ms	100 perintah lewat bridge
Worker, model uji ONNX	6.9 ms/frame	pra-proses + NMS di CPU
Publikasi deteksi	4.2 Hz	batas bawaan 5 Hz

9.2 Latensi REST



Masing-masing 300 permintaan berurutan.

9.3 Telemetri armada

20 bridge tiruan mengirim format Autel pada 10 Hz. Server menerima 2963 pesan; konsol menerima 1487 frame karena hub realtime sengaja menggabungkan data per drone ke laju

siaran 5 Hz (`GCS_TELEMETRY_BROADCAST_HZ`) agar peta tetap ringan. Tidak ada data yang hilang di server: seluruh pesan diproses untuk deteksi kejadian dan perekaman.

9.4 Batasan pengukuran

- Model uji ONNX hanya mengukur overhead pipeline (pra-proses, NMS, publikasi). Waktu inferensi YOLO asli bergantung pada ukuran model dan perangkat (CoreML/MPS di M4, CUDA di NVIDIA).
- Latensi video WebRTC dan kualitas link radio drone tidak diukur di sini.

9.5 Mengukur ulang di server produksi

```
cd backend && . .venv/bin/activate
python tests/bench.py --api http://localhost:8080 --password <sandi-admin> \
  --drones 20 --hz 10 --seconds 15 --out bench-m4.json
```

Skrip membuat drone sementara `bench-00` dst. lalu menghapusnya kembali.

BAB 10

Pemecahan masalah

Gejala	Penyebab dan tindakan
Drone tidak muncul terhubung	IP/port di bridge salah, firewall macOS memblokir 8080, atau ID drone di bridge berbeda dengan di konsol (lihat kejadian *drone tidak dikenal* di Log).
Tombol aksi abu-abu *Kendali belum diizinkan*	Interlock di aplikasi bridge mati. DJI: nyalakan sebelum START BRIDGE. Autel: nyalakan lalu SAVE.
Perintah *timeout*	Bridge tidak menjawab dalam 10 s (misi 30 s): periksa koneksi Wi-Fi RC dan log aplikasi bridge.
DJI menolak *Aktifkan motor*	Perilaku benar: MSDK v5 tidak punya arm terpisah.
Lepas landas Autel ditolak	GPS belum valid, kalibrasi kompas/IMU diminta, atau drone tidak di darat. Periksa aplikasi Autel.
Misi ditolak	Melebihi batas (120 m, 1 km, 10 m/s, 99 titik) atau posisi drone belum diketahui. Pesan menyebut titik yang bermasalah.
Stream *Tidak ada siaran*	Jalur RTMP di perangkat berbeda dengan yang terdaftar, atau port 1935 terblokir.
Video hitam di konsol	WebRTC: isi `LAN_IP`, buka UDP 8189; dari luar LAN gunakan VPN.
`worker: null` di /health	Worker mati atau NATS tak terjangkau: `pm2 logs gcs-worker`.
Model gagal dimuat	Format tidak cocok dengan worker (CoreML hanya macOS), file rusak, atau paket `ultralytics` belum terpasang.
Caption tidak muncul	`ollama pull moondream`; caption hanya berjalan di mode fokus.
STT 503	faster-whisper belum terpasang di server; konsol memakai STT browser.

BAB 11

Fitur saat ini dan rencana

11.1 Tersedia di versi 0.3.0

Area	Fitur
Armada	Telemetri DJI M4T dan Autel EVO II V3, status link, kejadian lepas landas/mendarat/baterai/RC, rekaman penerbangan dan riwayat jejak.
Kendali	Lepas landas, mendarat, RTH, tahan, terbang ke titik, misi waypoint (DJI KMZ, Autel), arm/disarm (Autel), dengan interlock dan batas keselamatan.
Video	RTMP/RTSP/HLS/SRT masuk, WebRTC keluar, kamera tetap/CCTV, pemilih lensa DJI.
AI	YOLO multi-model, hitungan grup, video beranotasi, caption Moondream dengan terjemahan, tanya-jawab visual, STT.
Demo	Drone simulasi dengan misi dan drop payload lengkap.
Platform	Web dan desktop Tauri, mode simulasi tanpa server, peran pengguna, log audit, deploy lokal.

11.2 Rencana

Tahap	Fitur
Berikutnya	Payload drop nyata (DJI PSDK), MAVLink untuk PX4/ArduPilot, telemetri LITO X1 dan SIYI, video jarak jauh lewat TURN/VPN.
Lanjutan	Pengenalan wajah, pemetaan 3D, peta luring, integrasi CCTV ONVIF, OpenIPC/WFB-ng.

BAB A

Protokol bridge

A.1 Endpoint

URL	Kegunaan
`ws://<server>:8080/telemetry/<id>`	Bridge DJI (ID di jalur)
`ws://<server>:8080/ws/drone`	Bridge Autel/generik (`droneId` di setiap pesan)
`ws://<server>:8080/bridge/<id>`	Bridge generik dengan ID tetap

Tambahkan `?token=<GCS_BRIDGE_TOKEN>` bila token diaktifkan.

A.2 Pesan

```
bridge → server  {"type":"hello","droneId":"evo2-v3-001","vendor":"autel",
                  "capabilities":["telemetry","commands","mission"]}
server → bridge  {"type":"command","id":"8…f2a","command":{"type":"takeoff","altM":10}}
server → bridge  {"type":"mission","id":"11…b0","plan":{"name":"Uji","takeoffAltM":20,
                  "endAction":"rtl","waypoints":[{"lat":-6.2,"lon":106.84,"altM":30,
                  "speedMs":3,"holdS":0,"action":"none"}]}}
bridge → server  {"type":"ack","droneId":"evo2-v3-001","id":"8…f2a","accepted":true,
                  "message":"Lepas landas (EVO II naik ±1,2 m lalu hover)"}
bridge → server  {"type":"event","droneId":"evo2-v3-001","severity":"info",
                  "message":"Titik 2/3 tercapai"}
```

Permintaan dan jawaban dicocokkan lewat field `id`. Batas waktu jawaban: 10 s untuk perintah, 30 s untuk misi.

A.3 Telemetri

DJI (tanpa `type`, 10 Hz): `pos{lat,lon,alt}`, `att{pitch,roll,yaw}`, `vel{x,y,z}` (NED), `heading`, `flying`, `sats`, `battery`, `mode`, `motors`, `home{lat,lon}`, `ts`.

Autel (`{"type":"telemetry","droneId",...,"data":{...}}`, 5 Hz): `latitude`, `longitude`, `relativeAltitude`, `roll`, `pitch`, `yaw`, `groundSpeed`, `velocityZ`, `satelliteCount`, `gpsFixType`, `batteryPercentage`, `flightMode`, `homeLatitude`, `homeLongitude`, `rcSignalLost`, `connected`, dan lainnya.

A.4 Jenis perintah

`arm`, `disarm`, `takeoff{altM}`, `land`, `rtl`, `hold`, `set-mode{mode}`, `goto{lat,lon,altM}`, `mission-start`, `mission-pause`, `payload-release`, `set-lens{lens: rgb|thermal|split}`.

BAB B

Referensi

B.1 Variabel lingkungan API

Variabel	Bawaan
GCS_HOST	"0.0.0.0"
GCS_PORT	8080
GCS_LOG_LEVEL	"info"
GCS_DATABASE_URL	"postgresql://gcs:gcs@localhost:5432/gcs
GCS_NATS_URL	"nats://localhost:4222"
GCS_JWT_SECRET	""
GCS_JWT_TTL_HOURS	12
GCS_ADMIN_USERNAME	"admin"
GCS_ADMIN_PASSWORD	""
GCS_ADMIN_DISPLAY_NAME	"Administrator"
GCS_CORS_ORIGINS	(daftar)
GCS_BRIDGE_TOKEN	""
GCS_DJI_DEFAULT_DRONE_ID	"m4t-001"
GCS_AUTEL_ATTITUDE_UNIT	"deg"
GCS_AUTEL_VZ_DOWN_POSITIVE	True
GCS_TELEMETRY_BROADCAST_HZ	5.0
GCS_MEDIAMTX_API_URL	"http://localhost:9997"
GCS_MEDIAMTX_RTSP_URL	"rtsp://localhost:8554"
GCS_MEDIAMTX_WEBRTC_PUBLIC_URL	"http://localhost:8889"
GCS_MEDIAMTX_HLS_PUBLIC_URL	"http://localhost:8888"
GCS_MEDIAMTX_LOCAL_HOSTS	(daftar)
GCS_MODELS_DIR	"data/models"
GCS_MODEL_MAX_BYTES	1024 * 1024 * 1024
GCS_MODEL_LOAD_TIMEOUT_S	180.0
GCS_OLLAMA_URL	"http://localhost:11434"
GCS_VLM_MODEL	"moondream"
GCS_TRANSLATE_MODEL	""
GCS_VLM_TIMEOUT_S	120.0
GCS_STT_ENGINE	"faster-whisper"
GCS_STT_MODEL	"small"
GCS_STT_DEVICE	"cpu"
GCS_STT_COMPUTE_TYPE	"int8"
GCS_DEMO_ENABLED	True
GCS_SEED_DEMO_DRONE	True
GCS_DEMO_BASE_LAT	-6.2088
GCS_DEMO_BASE_LON	106.8456
GCS_SIM_TIME_SCALE	1.0

B.2 Variabel lingkungan worker

Variabel	Bawaan
GCS_WORKER_NATS_URL	nats://localhost:4222
GCS_WORKER_MODELS_DIR	data/models
GCS_WORKER_DETECTOR_BACKEND	auto | ultralytics | onnxruntime
GCS_WORKER_ANNOTATED_OUTPUT	true
GCS_WORKER_ENCODER	libx264 | h264_videotoolbox | h264_nvenc
GCS_WORKER_OUTPUT_MAX_WIDTH	1280
GCS_WORKER_OUTPUT_FPS_CAP	25
GCS_WORKER_DETECTION_HZ	5
GCS_WORKER_OLLAMA_URL	http://localhost:11434
GCS_WORKER_PROMPTS_FILE	prompts.default.toml

B.3 Endpoint API

Semua endpoint kecuali `/auth/login`, `/health`, dan endpoint bridge memerlukan header `Authorization: Bearer <token>` (WebSocket/SSE: `?access_token=`).

Metode	Jalur
GET	/streams
PUT	/streams/{stream_id}/model
PUT	/streams/{stream_id}/models
POST	/streams/focus
GET	/streams/{stream_id}/detections/latest
GET	/streams/{stream_id}/captions
GET	/models
POST	/models
PATCH	/models/{model_id}
DELETE	/models/{model_id}
POST	/models/{model_id}/load
POST	/models/{model_id}/unload
POST	/vlm/ask
POST	/stt/transcribe
POST	/auth/login
GET	/auth/me
POST	/auth/change-password
POST	/auth/logout
GET	/drones
POST	/drones
POST	/drones/test-link
GET	/drones/{drone_id}
PUT	/drones/{drone_id}
DELETE	/drones/{drone_id}
GET	/drones/{drone_id}/flights
POST	/drones/{drone_id}/commands
POST	/drones/{drone_id}/mission
GET	/cameras

Metode	Jalur
POST	`/cameras`
PUT	`/cameras/{camera_id}`
DELETE	`/cameras/{camera_id}`
GET	`/missions`
GET	`/missions/{mission_id}`
POST	`/missions`
PUT	`/missions/{mission_id}`
DELETE	`/missions/{mission_id}`
GET	`/drops`
POST	`/drops`
PUT	`/drops/{drop_id}`
DELETE	`/drops/{drop_id}`
POST	`/drops/{drop_id}/execute`
POST	`/drops/{drop_id}/cancel`
GET	`/events`
GET	`/health`
WS	`/ws/realtime`
WS	`/ws/drone`
WS	`/bridge/{drone_id}`
WS	`/telemetry`
WS	`/telemetry/{drone_id}`

B.4 CLI administrasi

```
gcs-admin create-user --username <u> --display-name <nama> \
    --role admin|operator|observer [--unit <satuan>]
gcs-admin set-password --username <u>
```